

FTC-final

FELIPE VANDEVELDE - 72301201

RAMONYS SANTOS - 72301104

ARTUR PIUMBINI - 72400609

Junho de 2026

1 Introdução

Este trabalho final da disciplina de Fundamentos de Teoria da Computação (FTC) tem como objetivo estudar, especificar formalmente e implementar três modelos clássicos de computação: o Autômato Finito Determinístico (AFD), o Autômato com Pilha (APD) e a Máquina de Turing (MT). Esses modelos representam diferentes níveis de poder computacional e permitem reconhecer classes distintas de linguagens formais, desde linguagens regulares até linguagens recursivamente enumeráveis.

Assim, os objetivos específicos foram: (a) definir formalmente cada máquina; (b) demonstrar manualmente o processamento de cadeias de teste, evidenciando passo a passo as transições; (c) implementar simuladores funcionais, aderentes às definições formais; e (d) comparar os modelos, discutindo limitações e capacidades de reconhecimento de linguagens.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é definido formalmente por uma 5-tupla:

$$\text{AFD} = (Q, \Sigma, \delta, q_0, F)$$

Onde: Q é um conjunto finito de estados; Σ é o alfabeto de entrada; $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição determinística; $q_0 \in Q$ é o estado inicial; e $F \subseteq Q$ é o conjunto de estados de aceitação. Uma cadeia $w \in \Sigma^*$ é aceita se, após processar todos os símbolos de w a partir de q_0 , o autômato termina em um estado de F .

Exemplo de cálculo manual (modelo do projeto). No projeto, o AFD possui $\Sigma = \{a, b\}$, estado inicial q_0 e estado final q_2 , com transições (entre outras): $\delta(q_0, a) = q_1$; $\delta(q_0, b) = q_0$; $\delta(q_1, b) = q_2$; $\delta(q_2, b) = q_0$; etc.

Para a cadeia $w = \text{"ab"}$:

- Estado inicial: q_0 ;
- Lê 'a': $\delta(q_0, a) = q_1$;
- Lê 'b': $\delta(q_1, b) = q_2$.

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$$

Como o estado final é q_2 , a cadeia “ab” é aceita.

Para a cadeia $w = “a”$:

- Estado inicial: q_0 ;
- Lê ‘a’: $\delta(q_0, a) = q_1$;

$$q_0 \xrightarrow{a} q_1$$

Como o estado final é $q_1 \notin F$, a cadeia “a” é rejeitada.

2.2 Autômato com Pilha (APD) (com suporte a não determinismo)

Um APD e um APND é definido por uma 7-tupla:

$$APD = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Onde: Q é um conjunto finito de estados; Σ é o alfabeto de entrada; Γ é o alfabeto da pilha; $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ é a função de transição, que pode consumir símbolo de entrada ou ε e alterar a pilha; q_0 é o estado inicial; Z_0 é o símbolo inicial da pilha; e F é o conjunto de estados de aceitação (ou, alternativamente, pode-se adotar aceitação por pilha vazia).

No projeto, a aceitação foi configurada como aceitação por pilha vazia (isto é, $F = \emptyset$ e a condição de aceitação ocorre quando a pilha esvazia ao final do consumo da entrada).

Exemplo de cálculo manual – linguagem $L_2 = \{a^n b^n \mid n \geq 1\}$. A estratégia clássica é: enquanto lê ‘a’, empilha marcadores; ao começar a ler ‘b’, desempilha um marcador por ‘b’; aceita se a entrada terminar e a pilha esvaziar (removendo Z_0 ao final via transição ε).

Para $w = “aabb”$:

- Início: $(q_0, aabb, Z)$
- Lê ‘a’ e empilha: (q_0, abb, ZA)
- Lê ‘a’ e empilha: (q_0, bb, ZAA)
- Lê ‘b’ e desempilha: (q_1, b, ZA)
- Lê ‘b’ e desempilha: (q_1, ε, Z)
- ε -transição removendo Z : $(q_1, \varepsilon, \varepsilon)$

Pilha vazia e entrada consumida $\Rightarrow$ aceita.

Exemplo de cálculo manual – linguagem $L_3 = \{w \in a, b^* \mid w = w^R, |w| \geq 1\}$ (palíndromos). O autômato reconhece palíndromos utilizando uma pilha para armazenar os símbolos lidos durante a primeira fase do processamento. No estado inicial, cada símbolo da entrada é empilhado. De forma não determinística, o autômato pode decidir iniciar a fase de comparação em qualquer instante. Essa mudança pode ocorrer após consumir um símbolo da entrada, caracterizando

o caso de palavras de tamanho ímpar, ou por meio de uma transição ε , sem consumir símbolo algum, caracterizando o caso de palavras de tamanho par.

No estado de comparação, cada símbolo lido deve corresponder ao símbolo armazenado no topo da pilha. Havendo correspondência, o símbolo do topo é desempilhado e o processamento continua. Caso contrário, aquele ramo do processamento é rejeitado. Como o autômato é não determinístico, vários ramos podem ser explorados simultaneamente, sendo suficiente que apenas um deles leve à aceitação.

Considere a palavra $w = \text{"abba"}$, que possui tamanho par. Durante o processamento no estado inicial, o autômato lê os símbolos 'a' e 'b', empilhando os marcadores correspondentes. Em seguida, um dos ramos possíveis realiza uma transição ε , passando para a fase de comparação sem consumir um novo símbolo da entrada. A partir desse ponto, o símbolo 'b' é lido e comparado com o topo da pilha, removendo 'B'. Em seguida, o símbolo 'a' é lido e comparado com o topo 'A', que também é desempilhado. Ao final da leitura, resta apenas o símbolo inicial da pilha Z_0 , que é removido por uma última transição ε .

Como toda a entrada foi consumida e a pilha tornou-se vazia, esse ramo do processamento aceita a palavra. Esse exemplo evidencia o papel do não determinismo na escolha do ponto de transição entre as fases de empilhamento e comparação, bem como a utilização das transições ε para tratar palavras de tamanho par e remover o símbolo inicial da pilha, realizando a aceitação exclusivamente por pilha vazia.

2.3 Máquina de Turing (MT)

Uma Máquina de Turing pode ser definida por uma 7-tupla:

$$MT = (Q, \Sigma, \Gamma, \delta, q_0, q_{aceita}, q_{rejeita})$$

Onde: Q é um conjunto finito de estados; Σ é o alfabeto de entrada (não contém $_$); Γ é o alfabeto da fita, com $\Sigma \subseteq \Gamma$ e $_ \in \Gamma$; $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição (escreve na fita e move o cabeçote para a esquerda L ou direita R); q_0 é o estado inicial; q_{aceita} é o estado de aceitação; e $q_{rejeita}$ é o estado de rejeição.

No projeto, a MT foi implementada como simulador com fita dinâmica, cabeçote móvel, histórico de passos e limite de passos para evitar laços infinitos durante a execução.

Exemplo de cálculo manual – linguagem $L_4 = \{a^n b^n c^n \mid n \geq 1\}$. A linguagem L_4 é reconhecida por uma Máquina de Turing definida formalmente pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{aceita}, q_{rejeita})$, onde:

- $Q = \{1, 2, 3, 4, 5, 6, 7, 8, rej\}$;
- $\Sigma = \{a, b, c\}$;
- $\Gamma = \{a, b, c, X, Y, _\}$;
- $q_0 = 1$;
- $q_{aceita} = 8$;
- $q_{rejeita} = rej$;
- δ é a função de transição mapeada na tabela abaixo.

Estado Atual	Símbolo Lido	Próximo Estado	Símbolo Escrito	Direção
1	a	2	Y	R
2	a	2	a	R
2	X	2	X	R
2	b	3	X	R
3	b	3	b	R
3	X	3	X	R
3	c	4	X	R
4	c	5	c	L
4	–	7	–	L
5	a	5	a	L
5	b	5	b	L
5	X	5	X	L
5	Y	6	Y	R
6	X	6	X	R
6	a	2	X	R
7	X	7	X	L
7	Y	8	Y	R

Para $w_1 = aaabbbccc$ (o estado atual está indicado antes do símbolo que o cabeçote lê):

Início: 1 [a] a a b b b c c c

⊢ Y 2 [a] a b b b c c c

⊢ Y a 2 [a] b b b c c c

⊢ Y a a 2 [b] b b c c c

⊢ Y a a X 3 [b] b c c c

⊢ Y a a X b 3 [b] c c c

⊢ Y a a X b b 3 [c] c c

⊢ Y a a X b b X 4 [c] c

⊢ Y a a X b b 5 [X] c c

⊢ ... ⊢ 5 [Y] a a X b b X c c (o estado 5 volta o cabeçote até o primeiro símbolo 'Y')

⊢ Y 6 [a] a X b b X c c

⊢ Y X 2 [a] X b b X c c

⊢ Y X a 2 [X] b b X c c

⊢ Y X a X 2 [b] b X c c

⊢ Y X a X X 3 [b] X c c

⊢ Y X a X X b 3 [X] c c

⊢ Y X a X X b X 3 [c] c

⊢ Y X a X X b X X 4 [c]

$\vdash Y X a X X b X 5 [X] c$
 $\vdash \dots \vdash 5 [Y] X a X X b X X c$ (o estado 5 volta o cabeçote até o primeiro símbolo 'Y')
 $\vdash Y 6 [X] a X X b X X c$
 $\vdash Y X 6 [a] X X b X X c$
 $\vdash Y X X 2 [X] X b X X c$
 $\vdash Y X X X 2 [X] b X X c$
 $\vdash Y X X X X 2 [b] X X c$
 $\vdash Y X X X X X 3 [X] X c$
 $\vdash Y X X X X X X 3 [X] c$
 $\vdash Y X X X X X X X 3 [c]$
 $\vdash Y X X X X X X X 4 [_]$
 $\vdash Y X X X X X X X 7 [X] _$
 $\vdash \dots \vdash 7 [Y] X X X X X X X _$ (o estado 7 volta o cabeçote enquanto lê 'X' até o 'Y')
 $\vdash Y 8 [X] X X X X X X _$ (A máquina encerra o processamento no estado 8)

Palavra aceita: O estado 8 é o estado de aceitação da MT aceitando a palavra de teste

Para $w_2 = abcc$ (o estado atual está indicado antes do símbolo que o cabeçote lê):

Início: $1 [a] b c c$
 $\vdash Y 2 [b] c c$
 $\vdash Y X 3 [c] c$
 $\vdash Y X X 4 [c]$
 $\vdash Y X 5 [X] c$
 $\vdash \dots \vdash 5 [Y] X X c$ (o estado 5 volta o cabeçote até o primeiro símbolo 'Y')
 $\vdash Y 6 [X] X c$
 $\vdash Y X 6 [X] c$
 $\vdash Y X X 6 [c]$
 $\vdash Y X X rej [c]$
 $\vdash Y X X rej [c]$ (A máquina encerra o processamento no estado rej)

Palavra rejeitada: O estado rej é o estado de rejeição da MT rejeitando a palavra de teste

Exemplo de cálculo manual – Máquina transdutora da função $f(n) = n + 1$. A máquina é definida formalmente pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{aceita}, q_{rejeita})$, onde:

- $Q = \{q_0, q_1, rej\}$;
- $\Sigma = \{1\}$;
- $\Gamma = \{1, _\}$;

- $q_0 = 0$;
- $q_{aceita} = 1$;
- $q_{rejeita} = rej$;
- δ é a função de transição mapeada na tabela abaixo.

Estado Atual	Símbolo Lido	Próximo Estado	Símbolo Escrito	Direção
q0	1	q0	1	R
q0	–	q1	1	R

Para $w_1 = 111$ (o estado atual está indicado antes do símbolo que o cabeçote lê):

Início: $q_0 [1] 1 1$
 $\vdash 1 q_0 [1] 1$
 $\vdash 1 1 q_0 [1]$
 $\vdash 1 1 1 q_0 [–]$
 $\vdash 1 1 1 1 q_1 [–]$ (A máquina encerra o processamento no estado q1)

Palavra aceita: O estado q1 é o estado de aceitação da MT implementando a função n+1 na palavra de teste

Para $w_2 = 10$ (o estado atual está indicado antes do símbolo que o cabeçote lê):

Início: $q_0 [1] 0$
 $\vdash 1 q_0 [0]$
 $\vdash 1 rej [0]$ (A máquina encerra o processamento no estado rej)

Palavra rejeitada: O estado rej é o estado de rejeição da MT não implementando a função n+1 na palavra de teste

3 Descrição da Implementação

A implementação foi desenvolvida em C# com uma arquitetura de monólito modular, separando interface e módulos de domínio. A solução possui um projeto de entrada (UI em console) e três módulos independentes: Parte1 (AFD), Parte2 (APD) e Parte3 (MT). A execução é centralizada no módulo de UI, que oferece ao usuário um menu para selecionar a máquina a ser simulada.

3.1 Estrutura geral e decisões de projeto

A organização do repositório segue a divisão:

- Parte0: interface de terminal e orquestração do fluxo (menu, navegação entre telas);
- Parte1: implementação do AFD;

- Parte2: implementação do APD (com suporte a não determinismo);
- Parte3: implementação de uma MT com histórico e limite de passos.

Essa divisão favorece separação de responsabilidades, permitindo que cada máquina evolua de forma independente, com integração simples via rotinas públicas chamadas a partir do menu.

A interface utiliza Spectre.Console para melhorar a usabilidade no terminal, tornando o projeto mais consistente como simulador didático (menu, telas e retorno visual das execuções).

Em todos os módulos, a interface permite a entrada da palavra vazia (ε): basta pressionar Enter sem digitar qualquer caractere. Nesse caso, a cadeia recebida pelo simulador é a cadeia vazia (em C#), que corresponde formalmente a $\varepsilon \in \Sigma^*$. A opção `AllowEmpty()` do `Spectre.Console` garante que o campo de entrada aceite submissões vazias, delegando integralmente a semântica de ε à camada de domínio de cada módulo, conforme detalhado nas subseções a seguir.

3.2 Implementação do AFD (Parte1)

O AFD foi implementado com: conjunto de estados Q como vetor de cadeias de caracteres; alfabeto Σ como vetor de caracteres; e função de transição δ como `Dictionary<(string, char), string>`. Essa estrutura corresponde diretamente à definição formal $\delta : Q \times \Sigma \rightarrow Q$, permitindo consulta eficiente por par (estado, símbolo). O processamento de uma cadeia percorre seus símbolos, atualizando o estado corrente conforme δ . A aceitação verifica se o último estado após consumir toda a entrada pertence ao conjunto de aceitação (no projeto, um estado final único).

Também foram previstas rotinas auxiliares para retornar informações do autômato, listar transições para exibição e obter a sequência de estados visitados, facilitando depuração e documentação.

Tratamento de ε no AFD. Quando a entrada é a cadeia vazia, o laço `foreach` em `ObterTransicoes` não executa nenhuma iteração, pois não há símbolos a consumir. O vetor de transições retornado contém apenas o estado inicial q_0 . Como $q_0 \notin F$ (o único estado de aceitação é q_2), o método `Aceitar` verifica que o último estado do vetor não é o estado final e rejeita a palavra vazia.

Tratamento de símbolos fora do alfabeto no AFD. O alfabeto de entrada do AFD é $\Sigma = \{a, b\}$. Quando a cadeia contém um símbolo diferente de a ou b, não há transição definida no dicionário para o par (estado atual, símbolo lido). Nesse caso, o método `ObterTransicoes` acrescenta o estado auxiliar q_{-1} ao caminho de execução. Como q_{-1} não pertence ao conjunto de estados de aceitação, o método `Aceitar` rejeita a cadeia. Assim, entradas com símbolos fora de Σ são tratadas como transições para um estado de erro.

3.3 Implementação do APD (Parte2)

O APD foi estruturado explicitamente na forma da 7-tupla, com propriedades que representam: estados (Q) como lista de objetos `Estado`; alfabeto de entrada (Σ) como lista de `char`; alfabeto da pilha (Γ) como lista de `char`; e a função de transição (δ) como dicionário cuja chave é (estado, símbolo de entrada ou λ , topo da pilha) e cujo valor é uma lista de ações possíveis.

O uso de uma lista de transições por chave implementa diretamente o não determinismo: para uma mesma configuração, pode existir mais de uma transição, e o simulador deve explorar alternativas. A pilha foi implementada com `Stack<char>`, e a execução foi definida por recursão com clonagem da pilha a cada ramificação (cada caminho de computação mantém seu próprio conteúdo de pilha). Isso é essencial para simular corretamente um APND: escolhas diferentes não podem compartilhar a mesma instância de pilha.

O simulador também imprime configurações instantâneas no formato (estado, entrada restante, pilha), aproximando a execução prática da notação teórica. Note-se que o estado q_M (“estado morto”) é um estado auxiliar de sinalização utilizado exclusivamente na camada de apresentação para indicar ramos de computação rejeitados; ele não faz parte da 7-tupla formal do autômato, sendo criado localmente durante a execução do simulador.

Foram incluídos dois conjuntos de transições:

- $L_2 = \{a^n b^n \mid n \geq 1\}$, empilhando marcadores ao ler a, desempilhando ao ler b e aceitando por pilha vazia;
- L_3 (palíndromos em $\{a, b\}$ com $|w| \geq 1$), utilizando não determinismo para escolher o centro da palavra. Se o comprimento da palavra é par, o centro é escolhido por transições ε (representadas como $\backslash 0$ no código); se o comprimento é ímpar, a mudança de modo ocorre após consumir o símbolo central, seguida pela fase de comparação com desempilhamento.

Tratamento de ε no APD. Quando a entrada é a cadeia vazia, a posição de leitura já é igual ao comprimento da cadeia (ambos zero) na primeira chamada recursiva de `ExecutarAPND`. A pilha, porém, ainda contém o símbolo inicial Z_0 . Para $L_2 = \{a^n b^n \mid n \geq 1\}$, não existem transições normais (pois não há entrada), e a única λ -transição disponível em $(q_0, \backslash 0, Z)$ não esvazia a pilha de forma completa sem consumir entrada válida. Para L_3 (palíndromos com $|w| \geq 1$), o mesmo ocorre: nenhum ramo não determinístico consegue esvaziar a pilha sem que haja símbolos de entrada a comparar. Em ambos os casos, a palavra vazia é rejeitada.

Tratamento de símbolos fora do alfabeto no APD. O alfabeto de entrada configurado no APD é $\Sigma = \{a, b\}$. O simulador não interrompe a execução por uma validação prévia da cadeia; em vez disso, o tratamento ocorre pela própria função de transição. Se o símbolo lido não pertence a Σ , não existe transição normal cadastrada para consumi-lo. O APND ainda pode explorar transições λ disponíveis naquela configuração, mas nenhum ramo consegue consumir o símbolo inválido. Consequentemente, todos os caminhos possíveis terminam em rejeição, seja por alcançar o estado morto q_M , seja por não conseguir esvaziar a pilha com toda a entrada consumida.

3.4 Implementação da Máquina de Turing (Parte3)

Diferente dos modelos anteriores (AFD e APD), a MT exige suporte para leitura e escrita irrestritas sobre uma fita com movimentações nas duas direções, idealizada como potencialmente infinita, mapeando o modelo clássico da computação.

Para simular esse comportamento de fita infinita de forma eficiente em C#, optou-se pela utilização de um dicionário indexado por inteiros (`Dictionary<int, Simbolo> fita`). Essa abordagem elimina restrições de redimensionamento e complexidades de custo computacional ao avançar para posições à esquerda do ponto inicial (gerando índices negativos naturalmente), além de otimizar a memória ao tratar os espaços em branco (`_`) de forma esparsa. O controle do cabeçote é gerenciado por uma variável inteira simples, cujo deslocamento é computado aritmeticamente a cada ciclo conforme as direções 'L' ou 'R' retornadas pela função de transição.

Para lidar com execuções não terminantes, introduziu-se a propriedade `LimitePassos` (padrão de 1000 iterações), que interrompe execuções presas em laços infinitos, desde que o limite esteja adequadamente dimensionado. Também foi adicionado o parâmetro `MTRconhecadora`, que define se, ao final da execução, deve ser apresentada somente a aceitação/rejeição ou se é necessário apresentar o resultado final da fita, como demonstrado na operação $f(n) = n + 1$. Por fim, a rastreabilidade do cabeçote é garantida pelo método `FormatarFita()`, que isola o caractere sob o cabeçote entre colchetes (ex: `a[b]c`). Esse histórico de configurações é enviado à camada de apresentação, onde a biblioteca

Spectre.Console gera tabelas estruturadas e coloridas no terminal para detalhar cada passo do processo computacional ao usuário.

Tratamento de ε na MT. Quando a entrada é a cadeia vazia, o laço de inicialização da fita (`for (int i = 0; i < entrada.Length; i++)`) não executa nenhuma iteração, resultando em uma fita completamente vazia. Na primeira iteração do laço principal, o cabeçote está na posição 0, que não possui símbolo mapeado no dicionário, de modo que o símbolo lido é tratado como branco ($_$). Na MT reconhecedora de $L_4 = \{a^n b^n c^n\}$, não existe transição definida para $\delta(1, _)$, fazendo com que a máquina seja imediatamente encaminhada ao estado de rejeição. Na MT transdutora de $f(n) = n + 1$, a transição $\delta(q_0, _) = (q_1, 1, R)$ está definida, de modo que a máquina escreve 1 na fita e aceita, produzindo o resultado $f(\varepsilon) = 1$.

Tratamento de símbolos fora do alfabeto na MT. Na MT reconhecedora de L_4 , o alfabeto de entrada é $\Sigma = \{a, b, c\}$; na MT transdutora de $f(n) = n + 1$, o alfabeto de entrada é $\Sigma = \{1\}$. A implementação inicializa a fita com os caracteres digitados pelo usuário e a decisão de rejeição ocorre durante a consulta da função de transição. Se a máquina lê um símbolo fora do alfabeto esperado, não há transição cadastrada para o par (estado atual, símbolo lido). Nesse caso, a máquina leva ao estado de rejeição. Portanto, entradas com símbolos fora do alfabeto são rejeitadas por transição indefinida.

4 Resultados e Testes

Os testes foram organizados para validar o reconhecimento das linguagens-alvo por cada modelo. Para cada cadeia de teste, registrou-se o resultado (aceita/rejeita) e, quando aplicável, o caminho de computação (sequência de estados no AFD, configurações instantâneas no APD e histórico de fita/estado na MT).

4.1 Tabelas de resultados (modelo para preenchimento)

Observação: as cadeias do PDF não puderam ser extraídas a partir do conteúdo enviado; portanto, as tabelas abaixo estão prontas para serem preenchidas com os casos de teste do enunciado.

Nota sobre ε : nas tabelas a seguir, a entrada ε (palavra vazia) corresponde à submissão sem qualquer caractere digitado, isto é, o usuário pressiona Enter diretamente no prompt de entrada do simulador.

Tabela 1 – Resultados do AFD (Parte1)

Cadeia	Resultado obtido	Observação (sequência de estados)
ab	ACEITA	$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
aab	ACEITA	$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
bab	ACEITA	$q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
ababab	ACEITA	$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
ba	REJEITA	$q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1$
ε	REJEITA	$q_0 \xrightarrow{\varepsilon} q_{-1}$
b	REJEITA	$q_0 \xrightarrow{b} q_0$
ac	REJEITA	$q_0 \xrightarrow{a} q_1 \xrightarrow{c} q_{-1}$

Tabela 2 – Resultados do APD (Parte2) – $L_2 = \{a^n b^n \mid n \geq 1\}$

Cadeia	Resultado obtido	Observação (justificativa)
ab	ACEITA	$n = 1$
aabb	ACEITA	$n = 2$
aaabbb	ACEITA	$n = 3$
aab	REJEITA	2a, 1b
abb	REJEITA	1a, 2b
ba	REJEITA	ordem invertida
ac	REJEITA	símbolo c fora do alfabeto
ε	REJEITA	$n \geq 1$
abab	REJEITA	símbolos intercalados

Tabela 3 – Resultados do APD (Parte2) – L_3 (palíndromos)

Cadeia	Resultado obtido	Observação (justificativa)
a	ACEITA	palíndromo de tamanho ímpar; centro escolhido via transição com consumo
aba	ACEITA	palíndromo de tamanho ímpar; centro em 'b', ramo aceita por pilha vazia
abba	ACEITA	palíndromo de tamanho par; ε -transição escolhe centro entre 'b' e 'b'
ab	REJEITA	não é palíndromo; nenhum ramo leva à pilha vazia com entrada consumida
aab	REJEITA	não é palíndromo; todos os ramos falham na fase de comparação
aca	REJEITA	símbolo c fora do alfabeto; nenhum ramo consegue consumi-lo

Tabela 4 – Resultados da MT (Parte3)

Entrada	Resultado obtido	Observação (motivo/estado final)
abc	ACEITA	$[1, abc] \vdash^8 [8, YXX]$
aabbcc	ACEITA	$[1, aabbcc] \vdash^{24} [8, YXXXXXX]$
aaabbbccc	ACEITA	$[1, aaabbbccc] \vdash^{50} [8, YXXXXXXXXXX]$
aabbc	REJEITA	$[1, aabbc] \vdash^9 [rej, YabX]$
ab	REJEITA	$[1, ab] \vdash^4 [rej, YX]$
abd	REJEITA	$[1, abd] \vdash^4 [rej, YXd]$
abcabc	REJEITA	$[1, abcabc] \vdash^5 [rej, YXXabc]$
ε	REJEITA	$[1, _] \vdash^2 [rej, _]$

Tabela 5 – Resultados da MT (Parte3) – Transdutora $f(n) = n + 1$

Entrada	Saída (fita)	Observação (estado final)
1	11	$[0, 1] \vdash^3 [1, 11]$
111	1111	$[0, 111] \vdash^5 [1, 1111]$
ε	1	$[0, _] \vdash^2 [1, 1]$
10	REJEITA	$[0, 10] \vdash^2 [1, 10]$

5 Análise Comparativa

Em termos de poder computacional, existe uma hierarquia clássica entre os três modelos. O AFD reconhece exatamente as linguagens regulares, descritas por expressões regulares. Sua limitação fundamental é não possuir memória além do estado atual, portanto não consegue reconhecer padrões que exijam contagem arbitrária e comparação de quantidades, como $L_2 = \{a^n b^n\}$, que não é regular.

O APD amplia o poder computacional ao adicionar uma pilha, permitindo reconhecer as linguagens livres de contexto. Isso torna possível lidar com dependências do tipo “mesma quantidade” e estruturas aninhadas, como $a^n b^n$. Entretanto, o APD ainda possui limitações: há linguagens que não são livres de contexto, como $\{a^n b^n c^n\}$, que não podem ser reconhecidas por um APD de uma única pilha.

A Máquina de Turing é o modelo mais poderoso entre os três e pode simular AFDs e APDs, pois a fita atua como memória não limitada e a função de transição permite leitura, escrita e movimentação do cabeçote. Assim, a MT pode reconhecer linguagens que exigem múltiplas contagens, verificações mais complexas e processamento algorítmico geral, ultrapassando as limitações impostas pela finitude do AFD e pela memória restrita do APD.

Em termos de classes, tem-se a inclusão estrita:

$$\text{Regulares} \subset \text{Livres de Contexto} \subset (\text{reconhecíveis por MT}).$$

6 Conclusão

A execução deste projeto permitiu compreender de forma aplicada como definições matemáticas de máquinas abstratas se traduzem em estruturas de dados e algoritmos de simulação. Ao implementar um AFD, ficou evidente a simplicidade e a limitação do modelo, já que toda a “memória” do processo está contida no conjunto finito de estados. No APD, a introdução da pilha mostrou concretamente como uma memória auxiliar eleva significativamente o conjunto de linguagens reconhecíveis, além de destacar a importância do não determinismo para certas estratégias de reconhecimento, como no caso de palíndromos.

Por fim, a implementação da Máquina de Turing reforçou a ideia de computação geral e a necessidade de mecanismos de controle (como limite de passos) ao simular máquinas potencialmente não terminantes. Como aprendizado, o projeto consolidou a ligação entre teoria e prática: a formalização em tuplas, a noção de configuração instantânea e o raciocínio de aceitação/rejeição passaram a ser observáveis, testáveis e depuráveis em um simulador real.